

Security Audit Report

StakeDAO Migration Update for Concentrator by AladdinDAO



S E C B I T

June 30, 2026

1. Introduction

The AladdinDAO is a decentralized network to shift crypto investments from venture capitalists to the wisdom of crowds through collective value discovery. Concentrator is a yield-enhancing product launched by AladdinDAO, specifically designed for smart farmers who intend to leverage their liquidity provider (LP) assets to farm top-tier DeFi tokens at the highest possible annualized percentage yield (APY). Due to the adjustments made by the Stake DAO and Pendle protocols to their yield models—specifically transitioning from the ve-Token model to the vl-Token model—the Concentrator protocol is required to synchronize and update its corresponding codebase. SECBIT Labs conducted an audit from May 6 to June 30, 2026, including an analysis of the smart contracts in 3 areas: **code bugs**, **logic flaws**, and **risk assessment**. The assessment shows that the StakeDAO Migration Update for Concentrator has no critical security risks. The SECBIT team has some tips on logical implementation, potential risks, and code revisions (see Part 4 for details).

Type	Description	Level	Status
Design & Implementation	4.3.1 In the <code>VLSDTDelegation.boost()</code> function, the delegator and recipient parameters are erroneously inverted when calling <code>vlBoost.boost()</code> .	Medium	Fixed
Design & Implementation	4.3.2 When the delegation expiration time set by a user is not an exact multiple of a week, their expired shares will never be cleared from the system.	Medium	Fixed
Code Deployment	4.3.3 The <code>vLSDT</code> migration plan must explicitly clarify the compatibility of the legacy proxy storage and the transition scheme for the delegation address.	Medium	Discussed

2. Contract Information

This part describes the basic contract information and code structure.

2.1 Basic Information

The basic information about the StakeDAO Migration Update for Concentrator is shown below:

- Project website
 - <https://concentrator.aladdin.club/>
- Smart contract code
 - <https://github.com/AladdinDAO/aladdin-v3-contracts>
 - initial review commit [11bfd3f](#)
 - final review commit [8852c3c](#)

2.2 Contract List

The following content shows the contracts included in the StakeDAO Migration Update for Concentrator, which the SECBIT team audits:

Name	Lines	Description
SdPendleGaugeStrategy.sol	134	Together with the AutoCompoundingStrategyBaseV2 contract, it forms a complete contract where user deposits are funneled into the Stake DAO protocol. Additionally, this contract is responsible for managing the protocol's earnings.
EmergencyConverter.sol	27	In an emergency state, the administrator can switch to this contract, which halts all subsequent token swaps.
SdCRVBribeBurnerV2.sol	89	A contract that converts other earnings into sdCRV or CRV tokens and distributes them.
SdPendleBribeBurner.sol	80	This contract handles the bribe proceeds from Stake DAO protocol.
VISDTDelegation.sol	219	Users delegate their $v\text{LSDT}$ token voting power through this contract and receive the corresponding rewards.

3. Contract Analysis

This part describes code assessment details, including two items: "role classification" and "functional analysis".

3.1 Role Classification

There are two key roles in the StakeDAO Migration Update for Concentrator: Governance Account and Common Account.

- Governance Account
 - Description
Contract administrator
 - Authority
 - Update basic parameters
 - Transfer ownership
 - Method of Authorization
The contract administrator is the contract's creator or authorized by the transferring of the governance account.
- Common Account
 - Description
Users participate in the Concentrator.
 - Authority
 - boost some vlSDT token voting power to StakeDAOProxy
 - claim SDT token as reward
 - Method of Authorization
No authorization required

3.2 Functional Analysis

As a result of the yield model modifications implemented by the Stake DAO and Pendle protocols, the Concentrator protocol, which integrates with them, must correspondingly adjust its codebase logic. The SECBIT team conducted a detailed audit of some of the contracts in the protocol. We can divide the critical functions of the contract into two parts:

SdPendleGaugeStrategy

The strategy contract that connects the Concentrator and the Stake DAO protocol will facilitate the transfer of users' funds from this contract to the Stake DAO protocol. Simultaneously, the profits earned by users are withdrawn from this contract and reinvested. The main functions in these contracts are as follows:

- `deposit()`

This function deposits the funds from the strategy into the Stake DAO protocol.

- `withdraw()`

This function withdraws sdPendle tokens from Stake DAO and sends them to the specified address.

- `harvest()`

This function assists users in claiming their earnings, converting them back into sdPendle tokens, and then staking them into the Stake DAO protocol.

VISDTDelegation

Through this contract, users can delegate the voting power of their vLSDT holdings to the StakeDAOProxy contract. Concurrently, they are eligible to receive SDT token rewards. The main functions in these contracts are as follows:

- `boost()`

Users delegate the voting power of their vLSDT tokens to the StakeDAOProxy contract.

- `checkpointReward()`

Anyone can invoke this function to update the SDT token rewards under the contract.

- `claim()`

Users can claim SDT token rewards by this function.

4. Audit Detail

This part describes the process, and the detailed results of the audit also demonstrate the problems and potential risks.

4.1 Audit Process

The audit strictly followed the audit specification of SECBIT Lab. We analyzed the project from code bugs, logical implementation, and potential risks. The process consists of four steps:

- Full analysis of contract code line by line.
- Evaluation of vulnerabilities and potential risks revealed in the contract code.
- Communication on assessment and confirmation.
- Audit report writing.

4.2 Audit Result

After scanning with `adelaide`, `sf-checker`, and `badmsg.sender` (internal version) developed by SECBIT Labs and open source tools including `Mythril`, `Slither`, `SmartCheck`, and `Securify`, the auditing team performed a manual assessment. The team inspected the contract line by line, and the result could be categorized into the following types:

Number	Classification	Result
1	Normal functioning of features defined by the contract	✓
2	No obvious bug (e.g., overflow, underflow)	✓
3	Pass Solidity compiler check with no potential error	✓
4	Pass common tools check with no obvious vulnerability	✓
5	No obvious gas-consuming operation	✓
6	Meets ERC20 standard	✓
7	No risk in low-level calls (call, delegatecall, callcode) and in-line assembly	✓
8	No deprecated or outdated usage	✓
9	Explicit implementation, visibility, variable type, and Solidity version number	✓
10	No redundant code	✓
11	No potential risk manipulated by timestamp and network environment	✓
12	Explicit business logic	✓
13	Implementation consistent with annotation and other info	✓
14	No hidden code about any logic that is not mentioned in design	✓
15	No ambiguous logic	✓
16	No risk threatening the developing team	✓
17	No risk threatening exchanges, wallets, and DApps	✓
18	No risk threatening token holders	✓
19	No privilege on managing others' balances	✓
20	No non-essential minting methods	✓
21	Correct managing hierarchy	✓

4.3 Issues

4.3.1 In the `VLSDTDelegation.boost()` function, the delegator and recipient parameters are erroneously inverted when calling `vlBoost.boost()`.

Risk Type	Risk Level	Impact	Status
Design & Implementation	Medium	Design logic	Fixed

Location

- [VLSDTDelegation.sol#L149](#)

Description

It is necessary to confirm whether the current functionality of the `boost()` function operates as follows: Users delegate their own voting power to the `StakeDAOProxy` contract for operational management. In return, users can claim the SDT token rewards accumulated under the `VLSDTDelegation` contract. Under the current codebase logic, the function first invokes `delegableBalance()` to retrieve the remaining available voting power of the caller. Subsequently, it calls `vlBoost.boost()` to process the operations associated with the `vlSDT` token voting power. Upon further examination of the specific implementation (refer to the source code at: <https://github.com/stake-dao/contracts-monorepo/blob/5468fa16c1536ddceb05b14ba4f16d60a163edac/packages/vlsdt/src/vlBoost.sol#L220>), it is observed that this underlying function is designed to delegate a specified amount of `vlSDT` voting power from the delegator to the recipient for a designated duration ending at `endtime`.

```
// @audit link: https://github.com/stake-dao/contracts-monorepo/blob/5468fa16c1536ddceb05b14ba4f16d60a163edac/packages/vlsdt/src/vlBoost.sol#L220
/// @notice Create a boost delegation from delegator to recipient.
/// @dev Flat boost model: exact amount delegated, expires at week-aligned endtime.
/// @param delegator Address delegating boost (must be msg.sender or approved operator).
/// @param amount Amount of boost to delegate.
/// @param endtime Expiry timestamp (will be rounded down to week boundary).
/// @param recipient Address receiving the delegated boost.
function boost(address delegator, uint256 amount, uint256 endtime, address recipient) external override {
    // 1. Validate caller permissions and basic inputs.
    require(amount != 0, ZERO_AMOUNT());
    require(recipient != address(0), ZERO_ADDRESS());
    require(recipient != delegator, SELF_DELEGATION());
    // @audit it is required that the caller (msg.sender) must either be the delegator themselves, or have been granted prior authorization by the delegator
    require(msg.sender == delegator || _operators[delegator][msg.sender], NOT_OPERATOR());

    // 2. Align the expiry to a week boundary and validate duration constraints.
    // Upper bound uses ceil-aligned timestamp so marketplace round-up endtimes
```



```

// at non-boundary timestamps don't exceed the max duration check.
uint256 alignedEndtime = (endtime / WEEK) * WEEK;
uint256 nextWeek = ((block.timestamp + WEEK - 1) / WEEK) * WEEK;
require(alignedEndtime > block.timestamp, INVALID_EXPIRY());
require(alignedEndtime <= nextWeek + MAX_DURATION_WEEKS * WEEK,
INVALID_EXPIRY());

// 3. Refresh delegator state, compute available voting power, and enforce
capacity.
(uint256 currentDelegated,) = _checkpointWrite(delegator);
uint256 votingPower = vlSDT.balanceOf(delegator);
if (currentDelegated > votingPower) currentDelegated = votingPower;

// @audit amount of delegations available for the delegate
uint256 available = votingPower - currentDelegated;
require(amount <= available, INSUFFICIENT_BALANCE());

// 4. Book delegated-out amount and register when it expires.
delegatedTotal[delegator] = currentDelegated + amount;
// @audit records the number of expired votes for a delegator at the
alignedEndtime
delegatedExpiring[delegator][alignedEndtime] += amount;

// 5. Book delegated-in amount for recipient with the same expiry bucket.
// @audit retrieves the active escrowed balance remaining for a recipient
(, uint256 currentReceived) = _checkpointWrite(recipient);
receivedTotal[recipient] = currentReceived + amount;
// @audit records the number of expired votes for a recipient at the
alignedEndtime
receivedExpiring[recipient][alignedEndtime] += amount;

// 6. Emit ERC20 Transfer events (burn from delegator, mint to recipient).
emit Transfer(delegator, address(0), amount);
emit Transfer(address(0), recipient, amount);

// 7. Emit the delegation event.
emit Boost(delegator, recipient, amount, alignedEndtime);
}

```

Comparison of the parameters actually passed by the `VlSDTDelegation` contract:

```

IStakeDAOvlBoost(vlBoost).boost(stakeDAOProxy, _amount, _endtime, msg.sender);

```

Consequently, the semantic meaning of this code is that a specified `_amount` of `vlSDT` voting power belonging to the `stakeDAOProxy` will be delegated to the `msg.sender` for a designated duration ending at `endtime`. This appears to be inverted relative to the actual business requirements and requires urgent verification.

```

function boost(
    uint256 _amount,

```

```

uint256 _endtime,
address _recipient
) public {
    require(_recipient != address(0), "recipient is zero address");
    // @audit retrieves the total available voting power currently remaining for the
    caller
    if (_amount == uint256(-1)) {
        _amount = IStakeDAOvlBoost(vlBoost).delegableBalance(msg.sender);
    }

    // @audit this needs to be checked
    IStakeDAOvlBoost(vlBoost).boost(stakeDAOProxy, _amount, _endtime, msg.sender);

    _boost(_amount, _endtime, _recipient);
}

```

Suggestion

Adjust the parameters passed to the `vlBoost.boost()` function as follows. Concurrently, it should be noted that users must first invoke the `setOperator` function within the `vlBoost` contract to authorize the `VLSDTDelegation` contract; otherwise, users will fail to successfully call the `boost` function under the `VLSDTDelegation` contract.

```

function boost(
    uint256 _amount,
    uint256 _endtime,
    address _recipient
) public {
    require(_recipient != address(0), "recipient is zero address");
    if (_amount == uint256(-1)) {
        _amount = IStakeDAOvlBoost(vlBoost).delegableBalance(msg.sender);
    }

    // @audit fixed
    IStakeDAOvlBoost(vlBoost).boost(msg.sender, _amount, _endtime, stakeDAOProxy);

    _boost(_amount, _endtime, _recipient);
}

```

Status

The development team acknowledged the issue and resolved it according to the recommendation in commit [9ea3769](#).

4.3.2 When the delegation expiration time set by a user is not an exact multiple of a week, their expired shares will never be cleared from the system.

Risk Type	Risk Level	Impact	Status
Design & Implementation	Medium	Design logic	Fixed

Location

- [VLSDTDelegation.sol#L237](#)

Description

When users delegate their voting power via the `boost()` function, they are required to specify a delegation expiration timestamp, `_endtime`. It should be noted that when the `vlBoost.boost()` function processes the `_endtime` parameter, it rounds the timestamp down to the nearest whole week and records the expiration amount of the delegated `vLSDT` tokens based on this rounded time.

```
function boost(address delegator, uint256 amount, uint256 endtime, address
recipient) external override {

    .....

    // 2. Align the expiry to a week boundary and validate duration constraints.
    //    Upper bound uses ceil-aligned timestamp so marketplace round-up endtimes
    //    at non-boundary timestamps don't exceed the max duration check.
    // @audit aligns with the nearest weekly interval (rounded down)
    uint256 alignedEndtime = (endtime / WEEK) * WEEK;
    uint256 nextWeek = ((block.timestamp + WEEK - 1) / WEEK) * WEEK;
    require(alignedEndtime > block.timestamp, INVALID_EXPIRY());
    require(alignedEndtime <= nextWeek + MAX_DURATION_WEEKS * WEEK,
INVALID_EXPIRY());

    // 3. Refresh delegator state, compute available voting power, and enforce
capacity.
    (uint256 currentDelegated,) = _checkpointWrite(delegator);
    uint256 votingPower = vLSDT.balanceOf(delegator);
    if (currentDelegated > votingPower) currentDelegated = votingPower;

    uint256 available = votingPower - currentDelegated;
    require(amount <= available, INSUFFICIENT_BALANCE());

    // 4. Book delegated-out amount and register when it expires.
    delegatedTotal[delegator] = currentDelegated + amount;

    // @audit record expired amount
    delegatedExpiring[delegator][alignedEndtime] += amount;

    .....
}
```

However, when recording the expiration amount of a user's delegation, the `VLSDTDelegation` contract fails to adjust the `_endtime` parameter. This oversight leads to a state mismatch between the two contracts, specifically affecting the `delegatedExpiring[_user][_endtime]` accounting parameter in the code below.

```
function boost(
    uint256 _amount,
    uint256 _endtime,
    address _recipient
) public {
    require(_recipient != address(0), "recipient is zero address");
    if (_amount == uint256(-1)) {
        _amount = IStakeDAOVlBoost(vlBoost).delegableBalance(msg.sender);
    }

    // @audit note that this function internally rounds the `_endtime` down to the
    // nearest week, utilizing it as the actual expiration timestamp for the vlSDT escrow
    IStakeDAOVlBoost(vlBoost).boost(stakeDAOProxy, _amount, _endtime, msg.sender);

    // @audit there is no weekly truncation handling applied to _endtime here
    _boost(_amount, _endtime, _recipient);
}

function _boost(
    uint256 _amount,
    uint256 _endtime,
    address _recipient
) internal {
    // initialize claim index
    if (claimIndex[_recipient] == 0) {
        claimIndex[_recipient] = (block.timestamp / WEEK) * WEEK;
    }

    // update global state
    _update(_amount, _endtime, address(0));

    // update user state
    _update(_amount, _endtime, _recipient);

    emit Boost(msg.sender, _recipient, _amount, _endtime);
}

function _update(
    uint256 _delegated,
    uint256 _endtime,
    address _user
) internal {
    Point memory p = boosts[_user];
    _checkpointWrite(_user, p);
}
```

```

p.delegated += uint128(_delegated);

// @audit directly uses _endtime as the delegation deadline without performing
any rounding-down operations
delegatedExpiring[_user][_endtime] += _delegated;
boosts[_user] = p;

if (p.timestamp % WEEK == 0) {
    historyBoosts[_user][p.timestamp] = p.delegated;
}
}

```

More critically, the subsequent `_checkpointRead()` and `_checkpointWrite()` functions can only fetch the expiring delegation amounts that fall precisely on weekly epoch timestamps when reading expiration data. Consequently, if a user supplies an `_endtime` parameter that is not a multiple of a full week during escrow creation, their delegated voting power will never expire. This permanently breaks the lifecycle logic and directly disrupts the accurate distribution of protocol rewards under the contract.

```

function _checkpointRead(address _user) internal view returns (Point memory) {
    .....

    // @audit rounds down to the nearest whole week
    uint256 ts = (p.timestamp / WEEK) * WEEK;
    for (uint256 i = 0; i < 255; i++) {
        // @audit the `ts` parameter is updated in multiples of a full week
        ts += WEEK;
        uint256 _delegatedExpiring = 0;
        if (ts > block.timestamp) {
            ts = block.timestamp;
        } else {
            // @audit it can only read the expiring escrowed amounts that are recorded
            at the exact weekly epochs
            _delegatedExpiring = delegatedExpiring[_user][ts];
        }

        p.delegated -= uint128(_delegatedExpiring);
        p.timestamp = uint128(ts);

        if (p.timestamp == block.timestamp) {
            break;
        }
    }
    return p;
}

function _checkpointWrite(address _user, Point memory p) internal {
    .....

```

```

// @audit rounds down to the nearest whole week
uint256 ts = (p.timestamp / WEEK) * WEEK;
for (uint256 i = 0; i < 255; i++) {
// @audit the ts parameter is updated in multiples of a full week
    ts += WEEK;
    uint256 _delegatedExpiring = 0;
    if (ts > block.timestamp) {
        ts = block.timestamp;
    } else {
        // @audit it can only read the expiring escrowed amounts that are recorded at
the exact weekly epochs
        _delegatedExpiring = delegatedExpiring[_user][ts];
    }

    p.delegated -= uint128(_delegatedExpiring);
    p.timestamp = uint128(ts);

    if (ts % WEEK == 0) {
        historyBoosts[_user][ts] = p.delegated;
    }

    if (p.timestamp == block.timestamp) {
        break;
    }
}
}
}

```

Suggestion

To maintain consistency with the expiration logic inside `vlBoost.boost()`, the `_endtime` parameter must be rounded down to the nearest weekly epoch, which is then used as the official deadline for recording the user's delegation.

Status

The development team acknowledged the issue and resolved it according to the recommendation in commit [d535051](#).

4.3.3 The **vLSDT** migration plan must explicitly clarify the compatibility of the legacy proxy storage and the transition scheme for the delegation address.

Risk Type	Risk Level	Impact	Status
Code Deployment	Medium	Design logic	Discussed

Location

- [VeSDTDelegation.sol#L65-L89](#)

- [VlSdtDelegation.sol#L64-L86](#)
- [ConcentratorStakeDAOGaugeWrapper.sol#L55-L87](#)
- [ConcentratorStakeDAO.ts#L149-L164](#)
- [SdCRVBribeBurnerV2.sol#L45-L71](#)
- [SdPendleBribeBurner.sol#L45-L71](#)

Description

This Pull Request introduces `VlSdtDelegation` to replace the original `VeSdtDelegation` boost accounting mechanism. However, the current PR does not update the existing deployment scripts and test suites, nor does it provide new migration scripts or fork migration tests. Therefore, prior to official production deployment, the specific migration path must be clarified, and the corresponding scripts and tests must be completed.

There are two major risk vectors that require prior confirmation:

1. If we follow StakeDAO's recommendation to reuse/upgrade the existing `VeSdtDelegation` proxy (`0x6037Bb1BBa598bf88D816cAD90A28cC00fE3ff64`), the new implementation will be storage-incompatible with the legacy implementation.
2. Alternatively, if a brand-new `VlSdtDelegation` address is deployed, all wrappers, burners, and deployment configurations dependent on the old delegation address must be updated synchronously; otherwise, booster fees and associated logic will remain stuck on the legacy delegation path.

Furthermore, the boost point calculation in the legacy `VeSdtDelegation` utilizes a linear decay model, where the `Point` struct stores the `bias`, `slope`, and `ts`, with `slopeChanges` tracking the slope reductions scheduled for each expiration timestamp:

```
// VeSdtDelegation
struct Point {
    // The bias for the linear function
    uint112 bias;
    // The slop for the linear function
    uint112 slope;
    // The start timestamp in seconds for current epoch.
    // `uint32` should be enough for next 83 years.
    uint32 ts;
}

/// @notice Mapping from user address to current updated point.
/// @dev The global information is stored in address(0)
mapping(address => Point) public boosts;

/// @notice Mapping from user address to boost endtime to slope changes.
/// @dev The global information is stored in address(0)
mapping(address => mapping(uint256 => uint256)) public slopeChanges;
```

In contrast, the new `VLSDTDelegation` shifts to a flat boost model, where the `Point` struct stores the delegated amount and timestamp, with `delegatedExpiring` tracking the specific boost amounts scheduled to expire at each timestamp:

```
// VLSDTDelegation
struct Point {
    // The amount of delegated boost.
    uint128 delegated;
    // The start timestamp in seconds for current epoch.
    // `uint32` should be enough for next 83 years.
    uint128 timestamp;
}

/// @notice Mapping from user address to current updated point.
/// @dev The global information is stored in address(0)
mapping(address => Point) public boosts;

/// @notice Mapping from user address to boost endtime to slope changes.
/// @dev The global information is stored in address(0)
mapping(address => mapping(uint256 => uint256)) public delegatedExpiring;
```

Although these two layouts appear superficially similar in their declaration order, their underlying semantics are entirely different. If the implementation of the legacy proxy is directly upgraded from `VeSDTDelegation` to the current `VLSDTDelegation`:

- The legacy `bias / slope / ts` values stored within the `boosts[user]` slot will be misinterpreted by the new code as `delegated` and `timestamp`.
- The legacy `slopeChanges[user][endtime]` entries will be misinterpreted by the new code as `delegatedExpiring[user][endtime]`.
- The legacy `historyBoosts` records, which currently store decayed `bias` values, will be treated by the new logic as flat `delegated` power during reward distributions.

This will corrupt the `totalSupply()`, `balanceOf()`, checkpointing, and SDT reward accounting, leading to potential miscalculations of both user boost power and claimable rewards. Furthermore, because the legacy proxy has already been initialized, these corrupted states cannot be reset simply by executing the new implementation's `initialize()` function.

On the other hand, the delegation address within the associated wrapper is defined as an `immutable` constructor parameter:


```

/// @notice The address of VeSDTDelegation contract.
address public immutable delegation;

constructor(
    address _gauge,
    address _locker,
    address _delegation
) {
    gauge = _gauge;
    stakingToken = ICurveGauge(_gauge).staking_token();
    locker = _locker;
    delegation = _delegation;
}

```

As seen in the original deployment scripts, the constructor parameter for `ConcentratorSdCrvGaugeWrapper` utilizes the `VeSDTDelegation.proxy` address:

```

await deployment.contractDeploy(
    "ConcentratorSdCrvGaugeWrapper.implementation",
    "ConcentratorSdCrvGaugeWrapper implementation",
    "ConcentratorSdCrvGaugeWrapper",
    [
        SdCrvGauge,
        deployment.get("ConcentratorStakeDAOLocker.proxy"),
        deployment.get("VeSDTDelegation.proxy"),
        deployment.get("StakeDAOBribeClaimer"),
    ]
);

```

This implies that if the team elects to deploy a brand-new `VeSDTDelegation` address, the existing wrappers holding the legacy `immutable delegation` reference cannot be switched to the new address via a standard setter function. Instead, these associated contracts must either be redeployed or replaced through alternative migration mechanisms. Otherwise, the SDT booster fees processed during routine harvests and bribe harvests may still route into the legacy delegation address.

Furthermore, the constructors for `SdCRVBribeBurnerV2` and `SdPendleBribeBurner` have already integrated the new `_delegator` parameter:

```

// SdCRVBribeBurnerV2
address private immutable delegator;

constructor(address _wrapper, address _delegator) {
    wrapper = _wrapper;
    delegator = _delegator;

    // approval
    IERC20(CRV).safeApprove(_wrapper, type(uint256).max);
    IERC20(sdCRV).safeApprove(_wrapper, type(uint256).max);
}

```

```
// SdPendleBribeBurner
address private immutable delegator;

constructor(address _compounder, address _delegator) {
    compounder = _compounder;
    delegator = _delegator;

    // approval
    IERC20(sdPENDLE).safeApprove(_compounder, type(uint256).max);
}
```

These constructor parameter changes must likewise be reflected in the final migration scripts and test suites. Otherwise, even if the core contract code itself is fully remediated, the actual deployment pipeline may still fail to cover the new `_delegator` parameters, the wrapper delegation address cutovers, and the post-migration reward flows.

Choosing an incorrect migration path could result in the following severe outcomes:

- If the legacy `0x6037...` proxy is upgraded directly: The historical `veSDT` boost/reward states will be misinterpreted by the new flat accounting logic. This will inevitably corrupt all subsequent `SDT` reward distributions.
- If a brand-new `VLSDTDelegation` is deployed instead: The existing wrappers and burners will continue to point to the legacy delegation contract. Consequently, the new delegator will fail to receive booster fees, and the boost incentives associated with `asdCRV` and `asdPENDLE` will not migrate as intended.
- Without dedicated migration scripts and mainnet fork migration tests: It remains exceptionally difficult to verify the alignment and consistency of the proxy upgrade, new deployment, wrapper/burner wiring, and post-migration reward flows prior to production execution.

Suggestion

The definitive migration path must be established first to ensure total alignment across contracts, scripts, tests, and deployment artifacts.

- Option A: Reusing the Legacy Proxy Address
(`0x6037Bb1BBa598bf88D816cAD90A28cC00fE3ff64`)

A dedicated upgrade script must be written to deploy a storage-compatible V2 implementation, executed via the `ProxyAdmin`. The new implementation must strictly preserve the legacy storage layout, appending the new `vLSDT` flat-accounting storage variables only at the end. Concurrently, a definitive cutover, claim, liquidation, or freezing strategy for the historical `veSDT` rewards must be formalized.

- Option B: Deploying a Fresh `VLSDTDelegation` Address

The team must synchronously redeploy or upgrade all wrappers, burners, and related upstream contracts that hold the legacy delegation `immutable` reference. This guarantees that all `SDT` booster fees and boost accounting route entirely into the new address.

Based on the finalized migration path, developers must supplement or update deployment scripts, fork tests, deployment JSONs, and Typechain typings. This ensures that all constructor parameters, delegator addresses, and post-deploy assertions align perfectly with the live production configuration. Execute a comprehensive migration test suite on a live mainnet fork. The test must, at a minimum, validate the proxy upgrade or fresh deployment, wrapper/burner wiring, booster fee routing, historical reward claims/cutovers, and a complete harvest/bribe burn lifecycle.

Status

The team clarified that a fresh `VLSDTDelegation` deployment path will be used. Detailed deployment runbook, Safe transactions, and production execution verification are outside the scope of this report.

5. Conclusion

After auditing and analyzing the StakeDAO Migration Update for Concentrator, SECBIT Labs found some issues to optimize and proposed corresponding suggestions, which have been shown above.

Disclaimer

SECBIT smart contract audit service assesses the contract's correctness, security, and performability in code quality, logic design, and potential risks. The report is provided "as is", without any warranties about the code practicability, business model, management system's applicability, and anything related to the contract adaptation. This audit report is not to be taken as an endorsement of the platform, team, company, or investment.

APPENDIX

Vulnerability/Risk Level Classification

Level	Description
High	Severely damage the contract's integrity and allow attackers to steal ethers and tokens, or lock assets inside the contract.
Medium	Damage contract's security under given conditions and cause impairment of benefit for stakeholders.
Low	Cause no actual impairment to contract.
Info	Relevant to practice or rationality of the smart contract, could possibly bring risks.

SECBIT Lab is devoted to constructing a common-consensus, reliable, and ordered blockchain economic entity.

 <https://secbit.io>

 audit@secbit.io

 [@secbit_io](https://twitter.com/secbit_io)